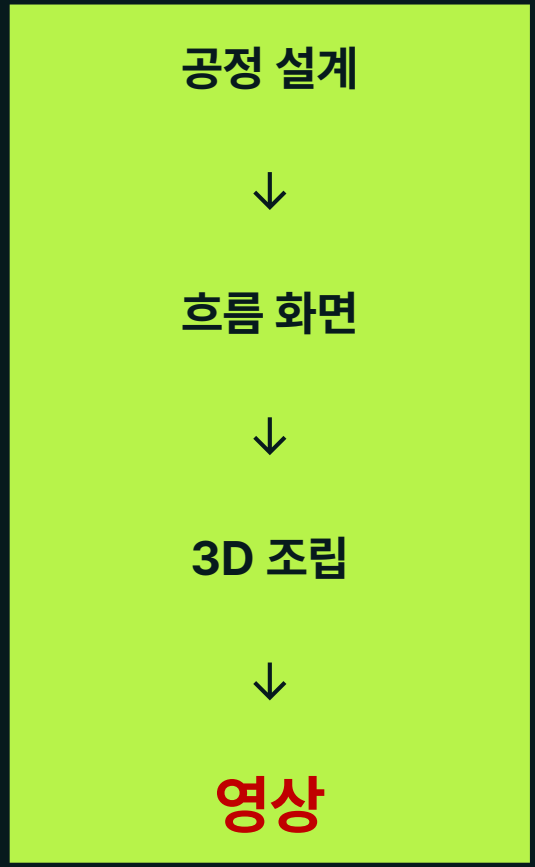
EV 공정 흐름 영상 만들기 초보자용 가이드

차가 공정을 따라 흘러가며 조립되는 영상을 프롬프트로 만듭니다



Day6 최종 프로젝트 · EV 전체 조립 모션그래픽

네 부분으로 나눠 열한 단계를 따라갑니다

01	공정 흐름 화면	**아이소메트릭 HTML** 공정 다섯 개 · 시간표 · 흐름 화면 · 속도 조절
02	3D 공정 흐름	**차체까지 든 GLB 모델** 부품 묶음 · 라인 장면 · 단계별 조립 · 카메라
03	영상으로 굽기	**프레임 캡처와 인코딩** 시험 프레임 판정 · 가상 시계 캡처 · MP4
04	점검과 문제 해결	**실제로 겪은 일** 점검표 · 멈춘 화면 · 가려진 차 · 느린 렌더

제자리에서 조립하는 영상이 아니라, 차가 라인을 따라 흘러가는 영상을 만든다

첫 번째 완성본은 공장 전체를 내려다보는 흐름 화면입니다

Car_Factory_Isometric_Flow.mp4 — 부품 투입 → 차체 조립 → 도장 → 최종 조립 → 검사·출하



※ 브라우저로 여는 HTML 한 장(Car_Factory_Isometric.html)을 그대로 녹화했다

두 번째 완성본은 차체까지 든 3D 모델이 라인을 지나갑니다

EV_Assembly_Line_Flow.mp4 — 프레임 → 배터리 → 구동계 → 차체 결합 → 검사·출하



※ 모델은 3_3D모델/EV_Crossover_Education.glb — 스케이트보드와 차체가 함께 들어 있다

먼저 다섯 개의 용어만 잡으면 됩니다

이 가이드 전체에서 반복해서 나온다

용어	뜻
공정 흐름	차 한 대가 스테이션을 차례로 지나며 조금씩 완성되는 모습
아이소메트릭	위에서 비스듬히 내려다본 평행 투영 — 공장 전체가 한눈에 보인다
GLB	3D 모델 파일 하나. 부품마다 이름이 붙어 있어 묶어서 움직일 수 있다
가상 시계	화면의 시간을 우리가 한 칸씩 넘기는 방식 — 느린 PC에서도 영상이 매끄럽다
인코딩	프레임 그림 수천 장을 MP4 한 파일로 묶는 일(ffmpeg)

영상의 품질은 공정 시간표가 정합니다 — 코드는 그 표를 따라 움직일 뿐입니다

준비물 세 가지만 사람이 확인합니다

1

Claude Code

데스크톱 앱 Code 탭

2

Chrome

화면 미리보기용

3

GLB 모델

3_3D모델 폴더

Blender가 없어도 된다 — 브라우저 3D(three.js)와 ffmpeg로 만든다

ffmpeg · 파이썬 도구 설치는 Claude Code에게 맡긴다

코드보다 공정 시간표를 먼저 받습니다

Claude Code에 붙여 넣을 프롬프트

자동차 조립 라인 영상을 만들려고 한다. 코드는 쓰지 말고 표부터 보여 줘.
· 공정 5개: 부품 투입 · 차체 조립 · 도장 · 최종 조립 · 검사·출하
· 공정마다 «이동 몇 초 → 작업 몇 초», 작업 중 화면에 보일 일 한 줄
· 전체 40~50초, 마지막엔 완성차가 출하 트레일러로 나간다
폴더 ev_flow를 만들고 이 표를 plan.md로 저장해 줘.

실행하면 이렇게 된다

채팅창	공정 · 이동 · 작업 시간 · 보일 일 표
생기는 파일	ev_flow/plan.md

확인: 표를 읽고 공정 순서와 시간을 먼저 확정한다 — 화면은 그다음이다

공장 전체를 내려다보는 흐름 화면을 만듭니다

Claude Code에 붙여 넣을 프롬프트

plan.md대로 index.html 한 파일에 아이소메트릭 공장 화면을 만들어 줘.
· SVG로 바닥 격자, 컨베이어, 공정 5곳(로봇 팔 포함), 출하 트레일러
· 차가 컨베이어를 따라 움직이고, 공정마다 멈춰 작업한 뒤 다음으로 간다
· 위쪽: 지금 공정 번호 · 이름 · 설명, 그 아래 공정 탭 5개와 진행 막대
· 아래쪽: 일시정지 · 처음부터 · 속도(0.5×~4×) · 반복 재생
외부 파일 없이 열리게 하고, 다 되면 Chrome으로 열어 줘.

실행하면 이렇게 된다

생기는 파일	ev_flow/index.html (혼자 열리는 HTML 한 장)
열면 보이는 것	차가 다섯 공정을 차례로 지나며 탭이 하나씩 켜진다

확인: 차가 제자리에 서 있지 않고 라인을 따라 흘러가는지 본다

공정마다 무엇이 바뀌는지 눈에 보이게 다듬습니다

Claude Code에 붙여 넣을 프롬프트

흐름 화면을 다듬어 줘.
· 공정마다 차 모습이 달라지게: 패널 → 차체 → 색 → 바퀴·유리 → 검사 표시
· 지금 작업 중인 공정은 테두리를 그 공정 색으로 빛나게
· 위쪽 설명 밑에 «도장 로봇이 상도를 분사합니다 (99%)»처럼 진행 문구
· 한 바퀴 돌면 «완성 차량» 숫자가 1씩 오른다

실행하면 이렇게 된다

화면	작업 공정이 색 테두리로 빛나고 진행 문구가 바뀐다
오른쪽 위	완성 차량 0 → 1 → 2 …

확인: 4× 속도로 한 바퀴를 보고 어색한 공정을 한 줄로 말해 고친다

3D 모델 안의 부품 이름부터 읽습니다

Claude Code에 붙여 넣을 프롬프트

3_3D모델/EV_Crossover_Education.glb 안을 읽어서 표로 보여 줘.
· 메시 개수, 이름 앞부분별 개수(Body · Battery · Front · Rear …)
· 길이 방향 축(x 또는 z)과 전체 크기
· 이 이름으로 네 묶음(프레임 · 배터리 · 구동계 · 차체)을 어떻게 나눌지 제안
파일은 고치지 말고 읽기만 해 줘.

실행하면 이렇게 된다

채팅창	메시 1,015개 — Body 607 · Front/Rear 380 · Battery 19 …
제안	Body_ → 차체, Battery_ · HV_ → 배터리, Front_ · Rear_ → 구동계

확인: 묶음 나누기가 실제 조립 순서와 맞는지 사람이 판정한다

브라우저 3D로 조립 라인 장면을 세웁니다

Claude Code에 붙여 넣을 프롬프트

ev_flow/line3d.html 을 three.js로 만들어 줘(three는 lib 폴더에 받아 둔다).
· 1920×1080, 어두운 바닥과 격자, 길게 이어진 컨베이어
· 13m 간격으로 스테이션 5곳: 뒤쪽 기둥 · 공정 색 램프 · 로봇 팔 2개
· 앞쪽 기둥은 두지 않는다(차를 가린다)
· 위쪽에 흐름 화면과 같은 공정 패널과 탭 5개
로컬 서버로 열어 GLB가 불러와지는지 보여 줘.

실행하면 이렇게 된다

생기는 파일	line3d.html · lib/three.module.js · GLTFLoader.js
화면	빈 라인 위에 차 모델이 팔레트에 실려 보인다

실제로 겪은 일: 앞쪽 기둥과 로봇 팔이 차를 가려 뒤쪽으로만 옮겼다

공정마다 부품 묶음이 하나씩 나타나게 합니다

Claude Code에 붙여 넣을 프롬프트

차가 스테이션을 지날 때마다 부품 묶음이 붙게 해 줘.
· ① 프레임: 레일 · 크로스멤버 · 트레이가 양옆에서 들어와 자리 잡는다
· ② 배터리: 모듈 18개가 위에서 차례로 내려온다
· ③ 구동계: 모터 · 서스펜션 · 바퀴가 옆에서 붙는다
· ④ 차체 결합: 차체가 한 덩어리로 위에서 내려와 맞물린다
· ⑤ 검사: 스캐너 고리가 차를 훑고 출하 트레일러로 나간다

실행하면 이렇게 된다

타임라인	이동 2.6초 → 작업 5.5초 반복, 전체 약 48초
화면	같은 차가 라인을 따라 가며 점점 완성된다

확인: 각 공정에서 «무엇이 새로 붙었는지» 한 문장으로 말할 수 있어야 한다

배터리 공정에서는 모듈이 위에서 차례로 내려와 깔립니다

② 배터리 탑재 — 공정 탭과 바닥 라벨이 지금 공정을 함께 가리킨다



카메라는 차를 따라가고 작업할 때만 다가갑니다

Claude Code에 붙여 넣을 프롬프트

카메라와 라벨을 다듬어 줘.
· 카메라는 비스듬히 위에서 차를 따라 옆으로 움직인다
· 작업 중일 때만 조금 다가갔다가, 이동할 때는 다시 물러난다
· 공정 이름 라벨은 위쪽 패널과 겹치지 않게 바닥 앞쪽에 둔다
· 마지막 3초는 «한 대 완성» 카드로 끝낸다
시험용으로 9개 시점(1·6·12·16·22·30·33·40·45초)의 그림을 저장해 줘.

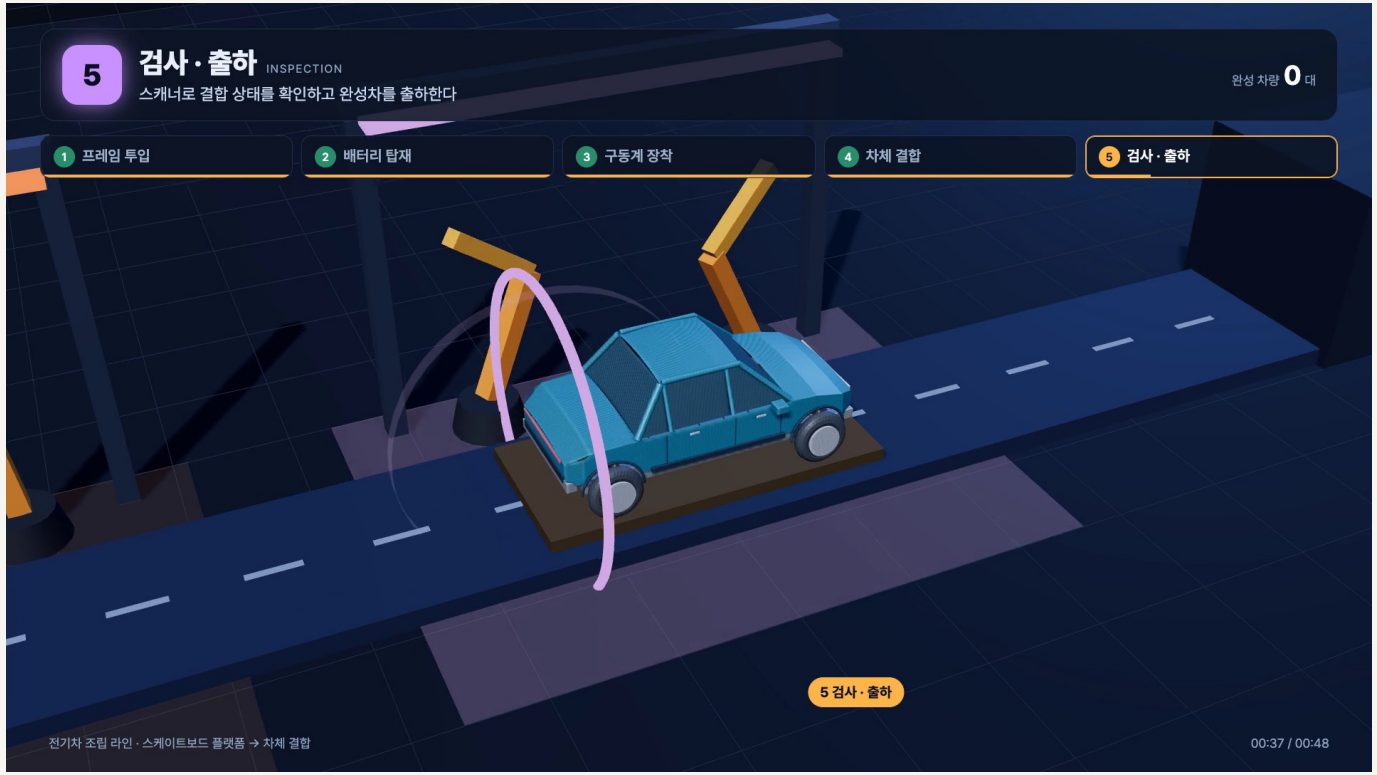
실행하면 이렇게 된다

생기는 그림	test/f00000.png ~ f00008.png 9장
채팅창	9장을 한 장에 모은 대조 시트

확인: 9장에서 차가 가려지거나 라벨이 패널에 겹친 장면을 찾아 말한다

마지막 공정에서는 스캐너 고리가 완성차를 훑습니다

⑤ 검사·출하 — 검사가 끝나면 출하 트레일러로 나가고 «완성 차량» 숫자가 오른다



시험 그림 9장을 사람이 판정합니다

볼 것	이러면 통과	아니면 이렇게 말한다
차가 보이나	모든 공정에서 차가 화면 가운데	«앞 기둥을 없애고 카메라를 높여 줘»
라벨	위쪽 패널과 겹치지 않는다	«라벨을 바닥 앞쪽으로 내려 줘»
새 부품	공정마다 새로 붙은 것이 보인다	«③에서 바퀴가 보이게 더 크게 해 줘»
끝 장면	«한 대 완성» 카드와 숫자 1	«완성 카드를 3초 보여 줘»

판정은 사람 몫 — 고칠 점은 한 번에 하나씩 말한다

가상 시계로 한 프레임씩 찍어야 영상이 매끄럽습니다

Claude Code에 붙여 넣을 프롬프트

두 화면을 영상용 프레임으로 찍어 줘(파이썬 playwright, 1920×1080, 30fps).
· 실시간 녹화는 하지 않는다. 화면의 시계를 우리가 1/30초씩 넘기고 한 장씩 찍는다
· 흐름 화면: requestAnimationFrame · performance.now를 가짜 시계로 바꾼다
· 3D 화면: renderAt(t) 함수를 열어 두고 t를 넘겨 가며 찍는다
· 아래쪽 조작 버튼은 숨기고, 한 바퀴(43초 · 48초)만 찍는다

실행하면 이렇게 된다

생기는 폴더	frames/ · f00000.png ~ (1,300~1,430장)
터미널	150장마다 진행 숫자

실제로 겪은 일: 그냥 녹화하면 느린 PC에서 시간이 늦게 흘러 6초에 첫 공정도 못 넘었다

프레임을 MP4 한 파일로 굽습니다

Claude Code에 붙여 넣을 프롬프트

frames 폴더를 MP4로 묶어 줘.
· ffmpeg, 30fps, H.264, crf 20, yuv420p, faststart
· 파일 이름: Car_Factory_Isometric_Flow.mp4 · EV_Assembly_Line_Flow.mp4
· 길이와 크기를 알려 주고, 3 · 14 · 31 · 37 · 44초 장면을 그림으로 보여 줘
다 되면 2_영상 폴더로 옮겨 줘.

실행하면 이렇게 된다

2_영상 폴더	EV_Assembly_Line_Flow.mp4 (48초 · 13MB)
	Car_Factory_Isometric_Flow.mp4 (44초)

확인: 장면 그림 다섯 장으로 처음 · 중간 · 끝이 모두 흐르는지 본다

필요하면 효과음과 자막을 마지막에 얹습니다

Claude Code에 붙여 넣을 프롬프트

EV_Assembly_Line_Flow.mp4에 소리를 얹어 줘(선택).
· 이동할 때 컨베이어 소리, 공정 작업 시작마다 짧은 «철컥» 효과음
· 차체가 맞물리는 순간(약 30초)에 조금 큰 효과음
· 배경음은 작게, 전체 -16 LUFS로 맞춘다
원본은 두고 _sound.mp4 로 따로 저장해 줘.

실행하면 이렇게 된다

생기는 파일	EV_Assembly_Line_Flow_sound.mp4
원본	소리 없는 판은 그대로 남는다

확인: 수업에서 틀 영상은 소리 있는 판, 발표에 끼울 영상은 소리 없는 판

다섯 가지가 되면 완성입니다

위에서부터 차례로 — 안 되면 다음 장의 문제 해결로

점검	이렇게 해 본다	이러면 통과
흐름	흐름 화면을 1×로 한 바퀴 본다	차가 다섯 공정을 차례로 지나간다
부품	3D 영상을 공정마다 멈춰 본다	공정마다 새 부품 묶음이 붙어 있다
가림	3D 영상 9개 시점을 본다	차가 기둥 · 팔에 가리지 않는다
매끄러움	영상을 처음부터 끝까지 튼다	끊김 없이 30fps로 흐른다
끝	마지막 3초를 본다	«한 대 완성»과 완성 차량 1

공정 순서가 실제 라인과 다르면 영상이 예뻐도 다시 만든다

막히면 증상을 그대로 Claude Code에 붙여 넣습니다

증상	원인	Claude Code에 붙여 넣을 말
차가 서 있기만 한다	작업만 있고 이동이 없음	공정 사이 이동 구간을 넣어 줘
녹화하면 너무 느리다	실시간 녹화	가상 시계로 한 프레임씩 찍어 줘
차가 기둥에 가린다	앞쪽 기둥 · 팔	앞 기둥을 없애고 팔은 뒤로 옮겨 줘
GLB가 안 열린다	파일로 바로 엶	로컬 서버를 켜고 그 주소로 열어 줘
라벨이 패널에 겹친다	라벨이 너무 높음	라벨을 바닥 앞쪽으로 내려 줘
Blender가 없다	설치 안 됨	Blender 없이 three.js로 만들어 줘

그 밖의 오류: 오류 문구를 그대로 붙이고 «원인을 설명하고 고친 다음 다시 실행해 줘»

열한 단계는 네 묶음입니다

한 묶음이 끝날 때마다 화면이나 그림을 열어 확인하고 다음으로 간다

STEP 01~03

공정 흐름 화면

- 시간표 먼저
- 흐름 화면
- 공정별 변화

STEP 04~07

3D 공정 흐름

- GLB 이름
- 라인 장면
- 부품 묶음

STEP 08

사람의 판정

- 시험 9장
- 가림 · 라벨
- 한 번에 하나

STEP 09~11

영상으로

- 가상 시계
- MP4
- 효과음

사람은 공정 순서와 판정만 — 만들고 굽는 일은 Claude Code가 합니다

차는 서 있지 않고 흘러갑니다.

공정마다 하나씩 붙습니다.

판정은 사람이 합니다.

같은 틀에 공정 시간표만 바꾸면
배터리 · 반도체 라인 영상도 됩니다.